

1 Data Preprocessing

1.1 What is Data Preprocessing?

Data preprocessing is the process of preparing raw data before it is used for data analysis or machine learning. In real-world situations, the collected data is rarely clean or perfectly structured. It may contain missing values, duplicate records, incorrect entries, inconsistent formats, or unnecessary information. Therefore, the data needs to be cleaned, checked, and transformed into a suitable form before it can be used effectively.

In simple words, data preprocessing means converting raw and imperfect data into clean, consistent, and meaningful data that can be easily understood by a machine learning model or used for further analysis.

Raw Data Problem	Example
Missing Value	Age is left blank
Duplicate Data	The same record appears more than once
Incorrect Value	Age = -5
Inconsistent Data	Kolkata, kolkata, KOLKATA
Wrong Data Type	Numbers stored as text
Outliers	An unusually high or low value

Table 1: Common Problems Found in Raw Data

1.2 Why is Data Preprocessing Needed?

Data preprocessing is important because the quality of the input data directly affects the quality of the analysis and machine learning results. A model trained on incomplete, inconsistent, or incorrect data may produce unreliable predictions even if the algorithm itself is well designed.

For example, suppose a dataset contains the location names “Kolkata”, “kolkata”, and “KOLKATA”. Although all three values refer to the same place, a computer may treat them as different categories. Similarly, a missing value or an invalid value can affect statistical calculations and may also reduce the effectiveness of a machine learning model.

Preprocessing helps solve these problems by making the data more reliable and consistent. It also ensures that the data is in a format that can be properly processed by machine learning algorithms.

The main reasons for performing data preprocessing are:

- To remove errors and inconsistencies from the dataset.
- To handle missing and duplicate values.
- To convert data into suitable formats and data types.
- To reduce the effect of abnormal or extreme values when necessary.

- To make different features comparable through suitable scaling.
- To create useful features that can improve the analysis or prediction.
- To ensure that the final dataset is suitable for machine learning.

Overall, data preprocessing acts as a bridge between raw data and a machine learning model. It transforms unorganized and imperfect data into a clean and usable dataset, which makes the later stages of analysis and model development more reliable.

1.3 Understanding Data Inspection

Data inspection is the first practical step in the data preprocessing process. Before making any changes to a dataset, it is important to understand what the data contains and whether there are any visible problems. This stage mainly focuses on examining the structure, size, data types, missing values, duplicate records, and general characteristics of the dataset.

In simple terms, data inspection can be considered as “looking at the data before cleaning it.” It helps us understand what is present in the dataset and what needs to be corrected in the next stages of preprocessing.

For example, consider a small demand dataset:

Region	Demand	Temperature	Date
Kolkata	120	31.2	01/09/2026
Kolkata	150	30.8	02/09/2026
Kolkata	NaN	31.0	03/09/2026
kolkata	180	NaN	04/09/2026

Table 2: Example of a Dataset Before Inspection

From the above dataset, some possible issues can already be noticed. The **Demand** column contains a missing value, the **Temperature** column also contains a missing value, and the values “Kolkata” and “kolkata” use different formats even though they represent the same region. The format of the **Date** column should also be checked to make sure that it is stored correctly.

For a large dataset, manually checking every record is not practical. Therefore, Python libraries such as Pandas are commonly used to perform the initial inspection.

1.3.1 Common Data Inspection Operations

The following operations are commonly used to understand a dataset before starting the cleaning process.

Operation	Purpose
<code>df.head()</code>	Displays the first few rows of the dataset and provides a quick view of the data.
<code>df.tail()</code>	Displays the last few rows and helps identify possible problems near the end of the dataset.
<code>df.shape</code>	Returns the number of rows and columns in the dataset.
<code>df.info()</code>	Provides information about column names, data types, and the number of non-null values.
<code>df.describe()</code>	Provides statistical information such as mean, minimum, maximum, and standard deviation for numerical columns.
<code>df.isnull().sum()</code>	Counts the number of missing values present in each column.
<code>df.duplicated().sum()</code>	Counts the number of duplicated records in the dataset.
<code>df.nunique()</code>	Shows the number of unique values available in each column.

Table 3: Common Data Inspection Operations

1.3.2 Purpose of Each Inspection Operation

head() The `head()` function displays the first five rows of the dataset by default. It is useful for getting a quick idea of how the columns and values are organized.

`df.head()`

The number of rows can also be specified, for example:

`df.head(10)`

shape The `shape` attribute returns the size of the dataset in the form:

(Number of Rows, Number of Columns)

For example, if the result is:

(50000, 12)

it means that the dataset contains 50,000 records and 12 attributes.

info() The `info()` function provides structural information about the dataset. It shows the column names, the number of non-null values, and the data type of each column.

For example, if a dataset contains 50,000 rows but the **Demand** column has only 49,850 non-null values, then:

$$50000 - 49850 = 150$$

Therefore, the **Demand** column contains 150 missing values.

`describe()` The `describe()` function provides a statistical summary of numerical columns. It usually includes the count, mean, standard deviation, minimum, maximum, and selected percentile values.

This is useful for understanding the general distribution of the data and for identifying values that may require further investigation. For example, if the average demand is around 150 units but the maximum demand is 900 units, the value 900 may be considered a possible outlier. However, it should not be removed automatically because it may represent genuine high demand during a festival or another special event.

`isnull().sum()` The following command is used to count missing values:

```
df.isnull().sum()
```

A possible output could be:

Region	0
Demand	150
Temperature	300
Festival	0
Date	0

This indicates that the `Demand` column has 150 missing values and the `Temperature` column has 300 missing values. The appropriate method for handling these values can then be decided during the data cleaning stage.

`duplicated().sum()` The following command checks for duplicate rows:

```
df.duplicated().sum()
```

If the result is 125, it indicates that 125 records are considered duplicates. However, duplicate-looking records should always be examined before removing them because similar records may represent different real-world events or transactions.

`nunique()` The `nunique()` function returns the number of unique values in each column.

```
df.nunique()
```

For example, if a project is expected to contain 40 regions but the `Region` column contains 57 unique values, the data should be checked for inconsistent naming or unexpected entries.

1.4 Example of Basic Data Inspection using Python

The following code can be used to perform a basic inspection of a CSV dataset.

```
import pandas as pd

df = pd.read_csv("data.csv")

print(df.head())
print(df.shape)
print(df.info())
print(df.describe())
print(df.isnull().sum())
print(df.duplicated().sum())
print(df.nunique())
```

These commands are primarily used for understanding the dataset. They do not normally change the original data. Instead, they help identify the problems that need to be addressed during the subsequent preprocessing stages.

1.5 Difference Between Data Inspection and Data Cleaning

Data inspection and data cleaning are related but they are not the same.

Aspect	Data Inspection	Data Cleaning
Main purpose	Understand the dataset and identify possible problems.	Correct or remove identified problems.
Main activity	Examining the data.	Modifying the data.
Examples	Checking missing values, data types, duplicates, and unusual values.	Filling missing values, removing duplicates, correcting formats, and fixing invalid entries.
Effect on data	Normally does not modify the dataset.	Changes the dataset to improve its quality.

Table 4: Difference Between Data Inspection and Data Cleaning

Therefore, data inspection can be viewed as the “understanding” stage, while data cleaning is the “correction” stage. Proper inspection should normally be performed before cleaning so that the cleaning process is based on the actual problems present in the dataset.

1.6 Typical Data Preprocessing Workflow

The complete preprocessing process can be represented as follows:

Raw Data → Data Inspection → Data Cleaning → Data Validation
→ Data Transformation → Feature Engineering → Feature Selection → Final Dataset

Each stage has a different purpose. Data inspection identifies the problems, data cleaning addresses those problems, validation checks whether the cleaned data follows the required rules, and transformation and feature engineering prepare the data for further analysis or machine learning.

1.7 Data Cleaning

After inspecting the dataset and identifying the possible problems, the next step is to clean the data. Data cleaning is the process of identifying and correcting missing, duplicated, incorrect, inconsistent, or irrelevant data.

In simple terms, data cleaning means fixing the problems found in the dataset so that the data becomes more accurate, consistent, and suitable for further analysis or machine learning.

For example, consider the following dataset:

Region	Demand	Temperature	Price
Kolkata	120	31.2	25000
kolkata	NaN	30.8	30000
Jadavpur	-20	NaN	"35,000"
Jadavpur	-20	NaN	"35,000"

Table 5: Example of a Dataset Containing Data Quality Problems

The above dataset contains several problems. The demand and temperature columns contain missing values, the region names are inconsistent, the demand value of -20 may be invalid, the price is stored in a text format, and the last two records are duplicates.

The main goal of data cleaning is not to remove as much data as possible. Instead, it is to preserve valid information while correcting genuine errors and making the remaining data more consistent.

1.7.1 Missing Values

A missing value occurs when information is not available for a particular record or attribute. Missing values may appear as `NaN`, `NULL`, an empty cell, or another placeholder.

For example:

Region	Demand	Temperature
Lake Gardens	120	31.2
Tollygunge	NaN	30.8
Ballygunge	145	NaN
Jadavpur	160	29.9

Table 6: Example of Missing Values

Missing values should not always be handled in the same way. The appropriate method depends on the type of data, the amount of missing information, and the reason why the value is missing.

Some common approaches are:

- Removing the record when only a small number of records are incomplete.
- Replacing a numerical value using the mean or median.
- Replacing a categorical value using the mode.
- Using a special value such as “Unknown” when appropriate.
- Using a more advanced statistical or machine learning method for imputation.
- Obtaining the correct value from another reliable data source.

For example, the median can be used to fill a missing numerical value:

```
df["Demand"] = df["Demand"].fillna(df["Demand"].median())
```

However, filling every missing value with a fixed number such as zero is not generally appropriate. A missing demand value does not necessarily mean that the actual demand was zero.

1.7.2 Duplicate Records

A duplicate record occurs when the same record is present more than once. Duplicate records may give unnecessary weight to the same observation and can affect the results of analysis.

Duplicate rows can be identified using:

```
df.duplicated().sum()
```

If the records are confirmed to be exact duplicates, they can be removed using:

```
df = df.drop_duplicates()
```

However, similar records are not always duplicates. For example, if the same customer places two orders on different dates, both records may be valid. Therefore, duplicate removal should be based on the meaning of the data and not only on similar values.

1.7.3 Incorrect Values

Incorrect values are values that do not follow the expected meaning or valid range of an attribute.

For example, consider the following demand values:

```
120
150
-20
170
```

If the **Demand** attribute represents the number of units requested or sold, a negative value may indicate a data entry or data processing error.

Such values can be identified using:

```
df[df["Demand"] < 0]
```

The appropriate action should then be decided after investigating the source of the error. The value may be corrected, treated as missing, or removed when it cannot be reliably corrected.

1.7.4 Inconsistent Values

Inconsistent values occur when the same information is represented using different formats.

For example, a region may be stored as:

```
Kolkata
kolkata
KOLKATA
Kolkata
```

Although these values refer to the same location, a computer may treat them as different categories.

Basic string operations can be used to make the values consistent:

```
df["Region"] = (
    df["Region"]
    .astype(str)
    .str.strip()
    .str.lower()
)
```

The resulting values can then be converted into the naming format required by the project.

For example:

```
df["Region"] = df["Region"].str.title()
```

This helps ensure that the same region is represented consistently throughout the dataset.

1.7.5 Incorrect Data Types

Another common problem is storing data using an inappropriate data type. Numerical values may be stored as strings, while dates may also be stored as ordinary text.

For example:

```
25000
30000
"35,000"
```

The value “35,000” may be interpreted as text because of the comma. This can cause problems during numerical calculations.

The value can be cleaned and converted into a numerical type:

```
df["Price"] = (
    df["Price"]
    .astype(str)
    .str.replace(",", "", regex=False)
)

df["Price"] = pd.to_numeric(
    df["Price"],
    errors="coerce"
)
```

Similarly, date values should be converted into a proper date format when they are needed for time-based analysis:

```
df["Date"] = pd.to_datetime(
    df["Date"],
    errors="coerce"
)
```

Correct data types make later calculations, filtering, grouping, and feature creation much easier.

1.7.6 Outliers

An outlier is an observation that is unusually different from the majority of the values in a dataset.

For example:

```
120, 135, 142, 150, 155, 900
```

The value 900 is much higher than the other observations and may therefore be flagged as a possible outlier.

However, an outlier is not automatically an error. In a supply-chain or demand forecasting dataset, a sudden increase in demand may be genuine due to a festival, holiday, promotion, weather condition, or another special event.

Therefore, the general process should be:

$$\text{Unusual Value} \rightarrow \text{Investigate} \rightarrow \begin{cases} \text{Valid Event} & \rightarrow \text{Keep} \\ \text{Data Error} & \rightarrow \text{Correct or Treat} \end{cases}$$

One commonly used method for detecting potential outliers is the Interquartile Range (IQR) method.

$$IQR = Q_3 - Q_1$$

The commonly used lower and upper boundaries are:

$$\text{Lower Bound} = Q_1 - 1.5(IQR)$$

$$\text{Upper Bound} = Q_3 + 1.5(IQR)$$

Values outside these limits can be flagged for further investigation.

1.7.7 Irrelevant Data

A dataset may sometimes contain attributes that are not useful for the particular analysis or prediction task.

For example, suppose a demand forecasting dataset contains:

```
Customer_ID
Region
Product
Demand
Temperature
Festival
Customer_Favorite_Color
```

If the objective is to predict product demand, `Customer_Favorite_Color` may not provide useful information for the model. Such attributes can be reviewed and removed when there is a clear reason for considering them irrelevant.

The decision should be based on the project objective rather than simply removing columns without analysis.

1.7.8 Practical Data Cleaning Process

The complete data cleaning process can be summarized as follows:

Raw Data → Identify Problems → Handle Missing Values
 → Remove Confirmed Duplicates → Correct Invalid Values
 → Standardize Inconsistent Values → Correct Data Types
 → Investigate Outliers → Clean Data

For example, consider the following raw data:

Region	Demand	Temperature	Price
Kolkata	120	31.2	25000
kolkata	NaN	30.8	30000
Jadavpur	-20	NaN	“35,000”
Jadavpur	-20	NaN	“35,000”

Table 7: Raw Data Before Cleaning

The following problems can be identified in the dataset:

- Missing values are present in the **Demand** and **Temperature** columns.
- The region names are not represented consistently.
- The demand value -20 may be invalid.
- The price value “35,000” is stored as text.
- The final two rows are exact duplicates.

After appropriate cleaning, the dataset should contain consistent formats, valid values, correctly typed attributes, and no unnecessary duplicate records.

1.8 Important Principle of Data Cleaning

Data cleaning should not be treated as a process of simply deleting every unusual or incomplete record. Every change should have a logical reason behind it.

For example, an unusually high demand value may initially look like an error, but it could represent a genuine increase during a major festival. Removing such a value without investigating it could result in the loss of important information.

Therefore, the main principle of data cleaning is:

Preserve valid information, correct genuine errors, and make the data consistent and reliable for further processing.

For a supply-chain demand forecasting system, proper data cleaning ensures that information such as demand, product details, prices, regions, weather attributes, dates, and festival information can be used reliably in the later stages of preprocessing and model development.

1.9 Data Validation

After the data has been cleaned, the next step is to verify whether the cleaned data is actually correct and follows the rules defined for the project. This process is known as data validation.

Data validation involves checking the values, formats, ranges, and relationships within the dataset to make sure that the data is logically correct and suitable for further processing.

In simple terms:

Data cleaning fixes the problems, while data validation checks whether the data is correct after cleaning.

For example, consider the following dataset:

Region	Demand	Price	Discount
Lake Gardens	120	50	10
Jadavpur	150	70	15
Garia	-20	60	10
Tollygunge	180	-50	120

Table 8: Example of Data Requiring Validation

Suppose the project defines the following rules:

- Demand must be greater than or equal to zero.
- Price must be greater than zero.
- Discount must be between 0 and 100.

Using these rules, the first two records satisfy all the conditions, while the third and fourth records contain values that require further investigation.

Therefore, validation helps ensure that the cleaned dataset is not only well-formatted but also logically correct.

1.9.1 Why is Data Validation Needed?

Cleaning the data does not automatically guarantee that every value is valid. A dataset may appear clean and organized while still containing values that violate the requirements of the project.

For example, the following values may be technically stored as valid numbers:

Demand = -50
Price = -100
Discount = 150

However, if the project rules do not allow these values, they should be identified during validation.

Data validation is therefore important for the following reasons:

- To ensure that the values fall within acceptable ranges.
- To check that required fields are present.
- To verify that data types and formats are correct.
- To ensure that values expected to be unique are not duplicated.
- To maintain consistency between related datasets.
- To identify records that violate business or project-specific rules.
- To reduce errors before the data is used for analysis or machine learning.

1.9.2 Types of Data Validation

Different types of validation can be applied depending on the nature of the dataset and the requirements of the project.

Range Validation Range validation checks whether a numerical value falls within an expected minimum and maximum range.

For example:

```
Demand >= 0
Price > 0
0 <= Discount <= 100
-90 <= Latitude <= 90
-180 <= Longitude <= 180
```

For example, a discount value of 120 would fail the following validation rule:

$$0 \leq \text{Discount} \leq 100$$

In Python, invalid discount values can be identified using:

```
invalid_discount = df[
    (df["Discount"] < 0) |
    (df["Discount"] > 100)
]
```

Data Type Validation Data type validation checks whether the values in a column are stored using the appropriate data type.

For example:

Attribute	Expected Data Type
Demand	Numeric
Price	Numeric
Date	Date/DateTime
Region	String
Product ID	String

Table 9: Expected Data Types for Common Attributes

The structure of a Pandas DataFrame can be inspected using:

```
df.info()
```

For example, a value such as “150 units” stored in the **Demand** column may need to be converted into a numerical representation before it can be used for calculations or machine learning.

Format Validation Format validation checks whether a value follows the required structure.

For example, the project may define the following formats:

```
Product_ID    -> P001
Warehouse_ID  -> WH001
Supplier_ID    -> SUP001
Date          -> YYYY-MM-DD
```

Values that do not follow the expected pattern can be flagged for further investigation.

For example:

```
WH001         -> Valid
WH015         -> Valid
WAREHOUSE5    -> Invalid
ABC           -> Invalid
```

Regular expressions and string operations can be used when more detailed format checking is required.

Uniqueness Validation Certain attributes are expected to contain unique values. These attributes may act as identifiers or primary keys within a dataset.

Examples include:

Product_ID
Warehouse_ID
Supplier_ID
Order_ID

Suppose the following values are found in a Product Master table:

P001
P002
P003
P003

If Product_ID is required to be unique, the repeated value P003 represents a validation problem.

This can be checked using:

```
df["Product_ID"].is_unique
```

Duplicate identifiers can also be located using:

```
df[df["Product_ID"].duplicated()]
```

Completeness Validation Completeness validation checks whether all required fields contain values.

For example, each demand record may be required to contain:

Region
Product_ID
Date
Demand

A record such as:

Lake Gardens | P001 | 2026-09-01 | NULL

would fail the completeness check because the demand value is missing.

The completeness of required columns can be checked using:

```
required_columns = [  
    "Region",  
    "Product_ID",  
    "Date",  
    "Demand"  
]  
  
print(df[required_columns].isnull().sum())
```

Date Validation Dates should be checked carefully when working with time-based datasets. The date should have a valid format and should also fall within the period defined for the project.

For example, if the project uses data from January 2024 to August 2026, a record outside this period may need to be investigated.

Dates can first be converted into a proper datetime format:

```
df["Date"] = pd.to_datetime(
    df["Date"],
    errors="coerce"
)
```

Invalid date values that cannot be converted are represented as missing values and can then be identified:

```
df[df["Date"].isna()]
```

The allowed date range can also be checked:

```
start_date = "2024-01-01"
end_date = "2026-08-31"

invalid_dates = df[
    (df["Date"] < start_date) |
    (df["Date"] > end_date)
]
```

Referential Integrity Validation Referential integrity validation is particularly important when multiple datasets are connected to each other.

For example, a demand record may contain a **Product_ID**. That product must exist in the Product Master table.

Similarly:

Demand.Product_ID → Product_Master.Product_ID

and:

Order.Warehouse_ID → Warehouse.Warehouse_ID

If a demand record contains:

Product_ID = P150

but the Product Master contains only P001 through P100, the record contains an invalid reference.

This type of validation helps maintain consistency between different tables and prevents broken relationships in the overall system.

Business Rule Validation Business rule validation checks whether the data follows rules defined by the actual application or business process.

For example, in a supply-chain system:

```
Demand >= 0
Supply_Quantity >= 0
Cost_Price > 0
Selling_Price > 0
0 <= Discount <= 100
```

A project may also define relationships between values. For example:

$$\text{Selling Price} \geq \text{Cost Price}$$

may be a required rule in a particular dataset if the system assumes that products are not sold below their recorded cost.

Another example is vehicle capacity. If a vehicle can carry only 100 units, an order requiring 500 units should be checked against the vehicle allocation and delivery rules.

These checks depend on the business logic of the specific project.

1.9.3 Validation in the Supply Chain Project

For the proposed intelligent multi-agent supply-chain system, validation should be performed across the different datasets used by the system.

Important validation rules may include:

Attribute / Relationship	Validation Rule
Demand	Demand must be greater than or equal to zero.
Supply Quantity	Supply quantity must be greater than or equal to zero.
Price	Cost price and selling price must be greater than zero.
Discount	Discount percentage must lie between 0 and 100.
Latitude	Latitude must lie between -90 and 90 .
Longitude	Longitude must lie between -180 and 180 .
Date	Date must be valid and fall within the defined project period.
Product ID	Product ID must exist in the Product Master table.
Warehouse ID	Warehouse ID must exist in the Warehouse table.
Supplier ID	Supplier ID must exist in the Supplier table.

Table 10: Example Validation Rules for the Supply Chain Dataset

These checks are useful because the system combines several related datasets, including demand, product, warehouse, supplier, weather, festival, and routing

information. An error in one dataset can affect the results produced by other components of the system.

1.9.4 Example of Validation

Consider the following data:

Region	Demand	Price	Discount
Lake Gardens	120	50	10
Jadavpur	150	70	15
Garia	-20	60	10
Tollygunge	180	-50	120

Table 11: Example Data for Validation

The validation rules are:

$$\text{Demand} \geq 0$$

$$\text{Price} > 0$$

$$0 \leq \text{Discount} \leq 100$$

The validation results can therefore be interpreted as follows:

Region	Demand	Price	Discount	Result
Lake Gardens	120	50	10	Valid
Jadavpur	150	70	15	Valid
Garia	-20	60	10	Invalid Demand
Tollygunge	180	-50	120	Invalid Price and Discount

Table 12: Validation Results

This example shows that validation does not necessarily change the data immediately. Instead, it identifies the records or values that do not satisfy the defined rules. These records can then be sent back for correction or additional investigation.

1.10 Difference Between Data Cleaning and Data Validation

Although data cleaning and data validation are closely related, they perform different tasks.

Aspect	Data Cleaning	Data Validation
Purpose	Fixes problems in the dataset.	Checks whether the dataset follows predefined rules.
Main question	“How can I fix this problem?”	“Is this data valid?”
Typical action	Fill missing values, remove duplicates, correct formats.	Check ranges, formats, uniqueness, and relationships.
Effect on data	Usually modifies the dataset.	Primarily verifies and flags invalid data.
Example	Convert “35,000” into 35000.	Check whether $\text{Price} > 0$.

Table 13: Difference Between Data Cleaning and Data Validation

The two stages can therefore be represented as:

Data Inspection $\rightarrow$ Data Cleaning $\rightarrow$ Data Validation

Data inspection identifies possible problems, data cleaning attempts to correct those problems, and data validation checks whether the resulting data satisfies the required conditions.

1.11 Important Principle of Data Validation

Validation rules should be defined according to the meaning of the data and the requirements of the project. A value should not be marked as invalid simply because it appears unusual.

For example, a sudden increase in demand during a major festival may be statistically unusual, but it may still be a valid observation. Therefore, validation should consider both the technical constraints of the dataset and the real-world context in which the data was generated.

The main objective of data validation is to ensure that the final dataset is logically consistent, follows the required rules, and can be safely passed to the next stage of preprocessing.

1.12 Data Transformation

After the data has been cleaned and validated, the next step is to transform the data into a suitable format for analysis or machine learning. Data transformation refers to the process of changing the representation, scale, structure, or format of the data while preserving the meaning of the original information.

In simple terms, data transformation means changing the form of valid data so that it can be used more effectively by statistical methods, machine learning algorithms, or other analytical processes.

For example, a price may initially be stored as:

"35,000"

After transformation, it can be represented as:

35000

The information remains the same, but the representation is now more suitable for numerical calculations.

1.12.1 Difference Between Data Cleaning and Data Transformation

Data cleaning and data transformation are closely related, but they have different purposes.

Aspect	Data Cleaning	Data Transformation
Main purpose	Identifies and corrects problems in the data.	Changes the representation of valid data into a more useful form.
Main question	“What is wrong with the data?”	“How should the data be represented?”
Examples	Handling missing values, removing confirmed duplicates, correcting invalid values.	Scaling numerical values, encoding categories, converting dates, and aggregating data.
Main objective	Improve data quality.	Improve data usability for analysis or modelling.

Table 14: Difference Between Data Cleaning and Data Transformation

Therefore, cleaning focuses mainly on correcting problems, while transformation focuses on preparing valid data in a representation that is appropriate for the next stage.

1.12.2 Why is Data Transformation Needed?

Raw data is not always stored in a form that can be directly used by a machine learning algorithm. Numerical variables may have very different scales, categorical values may be represented as text, and dates may contain several useful pieces of information that are not directly visible to the model.

For example, consider the following values:

Age = 25
Income = 500000
Demand = 1200

These variables represent different types of information and have very different numerical scales. Some machine learning algorithms can be affected by such differences in scale.

Similarly, categorical data such as:

Kolkata
Jadavpur
Garia

cannot normally be supplied directly to algorithms that require numerical input. Therefore, such values need to be transformed.

Data transformation helps in:

- Converting data into a suitable format.
- Making numerical features comparable when required.
- Converting categorical values into numerical representations.
- Extracting useful information from dates and time values.
- Reducing skewness in highly uneven numerical distributions when appropriate.
- Converting data from one level of aggregation to another.

1.12.3 Numerical Scaling

Numerical scaling changes the range or distribution of numerical variables. This can be useful when features have significantly different magnitudes and the selected machine learning algorithm is sensitive to feature scale.

Two commonly used approaches are standardization and normalization.

Standardization Standardization transforms a value using its mean and standard deviation:

$$z = \frac{x - \mu}{\sigma}$$

where:

- x is the original value,
- μ is the mean of the feature,
- σ is the standard deviation of the feature.

After standardization, the feature will generally have a mean close to zero and a standard deviation close to one.

In Python, standardization can be performed using:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

It is important to fit the scaler using the training data and then apply the same transformation to the validation or test data. This prevents information from the test set from influencing the preprocessing process.

Normalization A commonly used normalization method is Min-Max scaling.

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

This transformation generally maps the values to a range between 0 and 1.
For example:

10, 20, 30, 40, 50

can be transformed approximately into:

0.00, 0.25, 0.50, 0.75, 1.00

In Python:

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

The choice between different scaling methods depends on the characteristics of the data and the requirements of the selected algorithm.

1.12.4 Categorical Data Encoding

Categorical data contains labels or categories rather than numerical values.

For example:

Kolkata
Jadavpur
Garia

Many machine learning algorithms require numerical input. Therefore, categorical values may need to be converted into numerical representations.

One-Hot Encoding One-hot encoding creates a separate binary column for each category.

For example:

Kolkata	Jadavpur	Garia
1	0	0
0	1	0
0	0	1

Table 15: Example of One-Hot Encoding

One-hot encoding can be performed using:

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(
    handle_unknown="ignore"
)

encoded_data = encoder.fit_transform(
    df[["Region"]]
)
```

This method is useful when the categories do not have a natural order.

Ordinal Encoding Ordinal encoding can be used when categories have a meaningful order.

For example:

Low < Medium < High

can be represented as:

```
Low      = 1
Medium   = 2
High     = 3
```

However, ordinal encoding should not be used for categories that do not have a natural order. For example, assigning numerical values to “Kolkata”, “Delhi”, and “Mumbai” would incorrectly imply an ordering that does not exist.

1.12.5 Date and Time Transformation

Date values often contain more useful information than what is visible in the original date representation.

For example:

2026-09-17

can be transformed into:

Year
Month
Week
Day
Day of Week
Weekend Flag
Quarter

Using Pandas:

```
df["Date"] = pd.to_datetime(df["Date"])

df["Year"] = df["Date"].dt.year
df["Month"] = df["Date"].dt.month
df["Week"] = df["Date"].dt.isocalendar().week
df["DayOfWeek"] = df["Date"].dt.dayofweek
df["IsWeekend"] = (
    df["DayOfWeek"] >= 5
).astype(int)
```

This type of transformation is particularly useful for time-dependent problems such as sales and demand forecasting.

1.12.6 Logarithmic Transformation

Some numerical variables have highly skewed distributions, where a small number of observations are much larger than most of the remaining values.

For example:

10, 20, 30, 50, 100, 1000, 10000

A logarithmic transformation can reduce the effect of this large difference.

A commonly used transformation is:

$$x' = \log(1 + x)$$

In Python:

```
import numpy as np

df["Log_Demand"] = np.log1p(
    df["Demand"]
)
```

This transformation may be useful for variables such as demand, sales, income, or transaction values when the underlying distribution is strongly right-skewed.

1.12.7 Aggregation

Data transformation can also involve changing the level at which information is represented.

For example, daily demand may be aggregated into weekly demand:

$$\text{Weekly Demand} = \sum_{d=1}^7 \text{Daily Demand}_d$$

For example:

Monday = 20
Tuesday = 30
Wednesday = 25
Thursday = 35
Friday = 40

The total demand for the five recorded days is:

$$20 + 30 + 25 + 35 + 40 = 150$$

In a supply-chain application, data may be aggregated by:

Day
↓
Week
↓
Region + Product + Week

The appropriate level of aggregation depends on the objective of the analysis.

1.12.8 Binning or Discretization

Binning converts continuous numerical values into predefined groups or intervals.

For example, age values can be grouped as:

Age Range	Category
0–25	Young
26–40	Adult
41–60	Middle-Aged
61–100	Senior

Table 16: Example of Binning

In Python:

```
bins = [0, 25, 40, 60, 100]

labels = [
    "Young",
    "Adult",
    "Middle_Aged",
    "Senior"
]

df["Age_Group"] = pd.cut(
    df["Age"],
    bins=bins,
    labels=labels
)
```

Binning is useful when groups are more meaningful for the analysis than the original continuous values.

1.12.9 Data Transformation in the Supply-Chain Project

For the proposed supply-chain demand forecasting system, several transformations can be applied to prepare the data for machine learning.

Suppose a demand record contains:

```
Region      = Lake Gardens
Date        = 2026-09-17
Demand      = 250
Temperature = 31.5
Humidity    = 82
Festival    = Durga Puja
Weekend     = Yes
```

The data can be transformed as follows:

Original Attribute	Possible Transformation
Region	Encode categorical region information
Date	Extract year, month, week, and day of week
Demand	Apply suitable scaling or transformation when required
Temperature	Scale when required by the selected model
Humidity	Scale when required by the selected model
Festival	Convert categorical festival information into numerical representation
Weekend	Convert Yes/No into a binary value such as 1/0

Table 17: Examples of Data Transformation in the Supply-Chain Project

After transformation, a model-ready representation may contain numerical features such as:

Region Code	Week	Temperature	Humidity	Festival Code	Is Weekend
12	37	0.68	0.82	5	1

Table 18: Example of Transformed Features

The exact transformations used in the final model should be selected according to the characteristics of the data and the requirements of the machine learning algorithm.

1.12.10 Important Principle of Data Transformation

Data transformation should preserve the meaning of the original information while making the data easier to process and analyse. Not every dataset requires every type of transformation.

For example, scaling may be useful for some machine learning algorithms but unnecessary for others. Similarly, one-hot encoding may be appropriate for nominal categories, while ordinal encoding is more suitable for categories with a natural order.

Therefore, the transformation method should be selected according to the type of data, the purpose of the feature, and the requirements of the model.

The overall transformation process can be represented as:

Clean and Valid Data → Scaling → Encoding → Date Transformation

→ Aggregation → Other Required Transformations → Model-Ready Representation

Data transformation therefore acts as an important bridge between cleaned data and feature engineering. Once the data has been transformed into suitable representations, new and more meaningful features can be created from it in the feature engineering stage.